

Documentation of *syntheticTurbulenceInlet* – the new inlet boundary condition for OpenFoam-12 that generates synthetic turbulence

Hursanay Fyhn and Eirik Fyhn
*SINTEF Energy Research**
(Dated: April 14, 2025)

CONTENTS

I. Guide	1
A. Setup	1
B. Parameters	1
C. Example usage	2
II. Information	3
III. Test results	4

I. GUIDE

A. Setup

Here are the steps on getting the BC up and running on your local OpenFoam-12. Open a terminal and do the following:

1. Clone the repository to any folder (given that you have access) by typing:
`git clone git@gitlab.sintef.no:Hursanay.Fyhn/syntheticTurbulenceInlet.git`
2. Go into the cloned folder by typing
`cd syntheticTurbulenceInlet`
3. Source you local OpenFoam-12 the way you normally source it.
4. Compile the boundary condition and add it to the list of boundary conditions for your OpenFoam-12 by typing:
`wmake`
(ignore the message about .H files already existing)
5. You can now use the BC like any other, just remember to source the relevant library by adding this line to your controlDict:
`libs ("libsyntheticTurbulenceInlet.so");`
6. Within the cloned directory, you will find a folder titled tutorials. Try out the tutorial cases for the four different geometry modes inside the tutorials folder.

B. Parameters

- *massFlowMode* (bool) (optional)
Choose if you want to specify a massFlow (with true) or an Umean (with false).
Default: false

* hursanay.fyhn@sintef.no

- *massFlow* (scalar, table, coded or other Function1 types) (required if *massFlowMode* is true)
Total mass flow rate into the patch.
- *Umean* (scalar, table, coded or other Function1 types) (required if *massFlowMode* is false)
Mean speed into the patch.
- *geometryMode* (annulus, disk, rectangle, channel or arbitrary) (required)
Channel is rectangle with PBC in one direction. The purpose of the arbitrary mode is to generate turbulence on a patch with arbitrary shape, do not follow the DNS profiles unlike the other geometry modes due to constant U , R and L .
- *wallLongOrShort* (long or short) (optional)
When *geometryMode* is *channel*, choose if the wall is on the long or the short side.
Default: long
- *arbitraryModeRadius* (scalar) (optional)
When *geometryMode* is *arbitrary*, the user can choose the radius within which to generate turbulence.
Default: The largest distance from the patch center.
- *arbitraryModeCenter* (vector) (optional)
When *geometryMode* is *arbitrary*, the user can choose the center coordinate.
Default: geometrically averaged center coordinate of the patch.
- *Utau* (scalar) (optional)
Friction velocity
Default: U_{τ} is calculated from the expression for a smooth pipe. There is an absolute lower limit that maps the entire DNS profile over the entire width, and it can be forced by e.g. choosing a negative value.
- *swirlNr* (scalar) (optional)
Swirl number
Default: 0
- *d* (scalar) (optional)
Eddy density (ratio of the sum of eddy volumes to the eddy box volume)
Default: 1
- *scale* (scalar) (optional)
A scaling factor for U'
Default: 1
- *nCellPerEddy* (int) (optional)
Minimum eddy length in units of number of cells
Default: 1
- *Lref* (scalar) (optional)
Normalisation factor for integral-length scale
Default: 1
- *seed* (unsigned int) (optional)
Seed for the random numbers
Default: an arbitrary number

C. Example usage

```
inlet
{
type syntheticTurbulenceInlet;
geometryMode annulus;
massFlowMode true;
massFlow 2.53e-4;
}
```

II. INFORMATION

We have implemented a synthetic turbulence boundary condition for OpenFoam-12 inspired by the turbulent boundary condition called turbulentDFSEMInlet (referred to as DFSEM from here on) that exists in the ESI version and not in the foundation version of the OpenFoam.

There are some things in our implementation that are the same as in the paper by Poletto et. al. (referred to as the paper from here on), but different from DFSEM where there might have been errors in the implementation. First, we fixed the error where the normalization step was done before the application of the rotation matrix for Reynolds stress tensor R , so that it matches the paper. We fixed this error which made the eddies elongated in the right direction, the flow direction. Second, we changed the expression for U' which is the fluctuation in the velocity U . The functional form of the expression for U' , in other words, the dependence of U' on the radius of the eddy, is the same as in the paper. In DFSEM, this was different than the expression in the paper.

In addition, in our implementation, there are things that are different and improved from both the DFSEM and the paper. Recalculating the expressions in the paper gave us different scaling factors in the expression for U' . The new expression gave largely improved results.

Determined to make our BC more user friendly, we have implemented several features that did not exist in DFSEM. Firstly, we have implemented four of the most common geometries for the inlet that the user can choose between: an annulus, a disk, a rectangle and a channel. We have implemented automatic detection of the dimensions and the directions of the geometry of the inlet patch for these four modes. This mean the user does not need to specify the radii of the annulus and the disk, or the sides and the vertices of the rectangle and the channel. For the channel mode, the user has the freedom to choose if the longest or the shortest edge is the wall, the other edges will then be assumed to be periodic, also in terms of the eddies.

To use DFSEM, one needs to do some mapping and processing of the data beforehand. This is not necessary in our BC as we map the relevant information directly to the cells in OpenFoam. Our BC also works regardless of the flow direction because we use the normal vector of the patch to determine the direction and we compute the Reynolds stress tensor etc. correctly inside the boundary condition rather than from external files.

There is a need to generate random numbers for the position of the eddies and the direction of the rotation of the eddies. Random numbers in the code are generated with Mersenne Twister using the same seed over all the processors to make sure that the same random number is generated.

In the DNS data files, the quantities are given as a function of the position y or y^+ . y^+ is the dimensionless position that is scaled with friction velocity u_τ and the kinematic viscosity ν as $y^+ = u_\tau y / \nu$. We use y^+ instead of y in mapping of the DNS data onto the geometry, as it gives us more flexibility due to it being scaled with the boundary layer thickness. We do that by having friction velocity u_τ as a parameter. In the code, u_τ is set to have an absolute lower minimum limit, that will map the entire profile of the DNS data over the entire width. Larger u_τ makes the boundary layer thinner and the middle is filled in with a constant that is the half-width value from the data. u_τ can also be specified by the user, if not, it will be calculated by default by using the expression for a smooth pipe.

We have implemented the possibility to have a swirling velocity profile in the code. Swirl number S is defined as

$$S = \frac{\int_A \rho \bar{u}_x \bar{u}_\theta r dA}{\bar{r} \int_A \rho \bar{u}_x^2 dA} \quad (1)$$

ρ is the density, $\bar{u}_x$ is the mean velocity profile in the axial direction, $\bar{u}_\theta$ is the mean velocity profile in the angular direction, r is the radius, A is the area, and $\bar{r} = (r_i + r_o)/2$ where r_i is the inner radius, r_o is the outer radius. To have a $\bar{u}_\theta$ that follows the shape of $\bar{u}_x$ and a velocity profile that does not have a curl that diverges when $r_i = 0$, we choose a form

$$\bar{u}_\theta = f_\theta r \bar{u}_x \quad (2)$$

where f_θ is a constant. Inserting Eq. (2) into Eq. (1) gives

$$S = \frac{f_\theta \int_A \rho \bar{u}_x^2 r^2 dA}{\bar{r} \int_A \rho \bar{u}_x^2 dA} \quad (3)$$

In the code, for a user given S , f_θ is calculated from Eq. (3) and inserted into Eq. (2) to find $\bar{u}_\theta$.

We have also added the functionality to specify mass flow rate instead of a mean speed.

To summarize, the results produce a better match to the original DNS data than DFSEM due to the recalculated expression. Also, there are many additional features that have been added to our implementation that improves the ease of use.

III. TEST RESULTS

?? shows the time averaged quantities measured over a line at various positions x .